

E N T E R P R I S E A Z U R E S E C U R I T Y

ENVIRONMENT

Technical Implementation Report

Azure Landing Zone · Hub & Spoke Networking · Zero Trust Security
Key Vault · IAM · Azure Monitor · Cost Management · IaC

Diogo Sequeira · Cloud Security Engineer · June 2026 · Version 1.0

Table of Contents

3. Executive Summary

This report documents the design, implementation, and validation of an Enterprise Azure Security Environment — a production-grade cloud infrastructure built according to Microsoft's Landing Zone principles and Zero Trust security model. The project demonstrates applied competency in cloud architecture, network security, identity and access management, governance, and observability within the Microsoft Azure ecosystem.

The environment was deployed entirely within a single Azure subscription, structured around six dedicated Resource Groups, each responsible for a distinct functional domain. A Hub & Spoke network topology was implemented to centralise network connectivity and enforce traffic inspection, with separate Virtual Networks for hub, development, and production workloads. Security controls were applied at every layer of the architecture, from network-level filtering using Network Security Groups and Application Security Groups, to identity-based access control via Azure RBAC, and secret management through Azure Key Vault backed by Private Endpoints.

Governance policies were enforced via Azure Policy, ensuring consistent tagging, regional compliance, and resource configuration standards across the entire subscription. Monitoring and observability were established through a centralised Log Analytics workspace, with Diagnostic Settings routing activity logs, metrics, and security events from all services into a single data plane. Azure Monitor alerts were configured to trigger on cost thresholds and security anomalies. Cost management controls, including budgets and spending alerts, were implemented to ensure financial accountability throughout the project lifecycle.

The infrastructure was designed with automation in mind: all components were provisioned through the Azure Portal during the initial build phase, with full Infrastructure as Code conversion planned using Terraform. The resulting environment serves as both a functional security reference architecture and a portfolio artefact demonstrating enterprise-level cloud engineering competency.

Project at a Glance	
Cloud Provider	Microsoft Azure
Region	West Europe (westeurope)
Architecture Pattern	Hub & Spoke Landing Zone
Resource Groups	6 (network, security, monitoring, identity, dev, prod)
Security Model	Zero Trust — Network + Identity + Data
Identity & Access	Azure RBAC + Managed Identity + Key Vault RBAC
Secret Management	Azure Key Vault + Private Endpoint
Monitoring	Log Analytics Workspace + Azure Monitor + KQL
Cost Controls	Azure Budgets + Cost Alerts
Infrastructure as Code	Terraform (planned)
Project Duration	June 2026
Author	Diogo Sequeira — Cloud Security Engineer

4. Project Objectives

The primary goal of this project was to design and deploy a realistic, enterprise-grade Azure environment that reflects the practices and standards applied in professional Cloud Security Engineering roles. The following objectives guided the entire implementation:

4.1 Technical Objectives

- ▶ Design and deploy a Hub & Spoke network topology following Microsoft's Azure Landing Zone accelerator patterns
- ▶ Implement network security controls using NSGs, ASGs, and custom Route Tables to enforce traffic segmentation
- ▶ Configure centralised secret management using Azure Key Vault with Private Endpoint integration
- ▶ Apply the principle of least privilege through Azure RBAC role assignments and Managed Identities
- ▶ Enforce governance standards across the subscription using Azure Policy definitions and assignments
- ▶ Build a monitoring and observability layer using Log Analytics, Diagnostic Settings, and KQL queries
- ▶ Implement cost controls through Azure Budgets and automated cost alerts

4.2 Professional Objectives

- ▶ Produce a reference architecture suitable for cloud security job applications and technical interviews
- ▶ Demonstrate the ability to translate architectural decisions into documented, reviewable artefacts
- ▶ Create a foundation for Infrastructure as Code conversion using Terraform modules
- ▶ Build a GitHub portfolio repository with clean, professional documentation
- ▶ Generate a comprehensive technical report that any recruiter or hiring manager can evaluate independently

5. Solution Overview

The Enterprise Azure Security Environment is structured as a multi-tier cloud architecture deployed within a single Azure subscription. The design follows the Azure Landing Zone concept — a standardised, scalable foundation for deploying workloads in a secure, governed, and well-architected manner.

The solution is centred around six Resource Groups that separate concerns clearly: network infrastructure, security resources, monitoring, identity management, development workloads, and production workloads. All Resource Groups are deployed to the West Europe region, ensuring low latency and GDPR-compatible data residency.

5.1 Architectural Principles

- ▶ Separation of Concerns — each Resource Group is responsible for a single functional domain
- ▶ Defence in Depth — security controls applied at network, identity, and application layers
- ▶ Least Privilege Access — all identities operate under the minimum permissions required
- ▶ Observability by Default — all services emit logs and metrics to a central workspace
- ▶ Governance as Code — Azure Policy enforces compliance automatically
- ▶ Cost Visibility — spending is tracked and alerted at subscription level

5.2 Subscription Structure

The following structure represents the logical organisation of the subscription:

Azure Subscription 1

— rg-enterprise-network	→ Hub VNet, NSGs, Route Tables, Private Endpoints
— rg-enterprise-security	→ Key Vault, Microsoft Defender, Security Policies
— rg-enterprise-monitoring	→ Log Analytics Workspace, Azure Monitor, Alerts
— rg-enterprise-identity	→ Managed Identities, RBAC Assignments
— rg-enterprise-dev	→ Development VNet and workloads
— rg-enterprise-prod	→ Production VNet and workloads
— rg-secure-landing-zone	→ Secure Landing Zone components

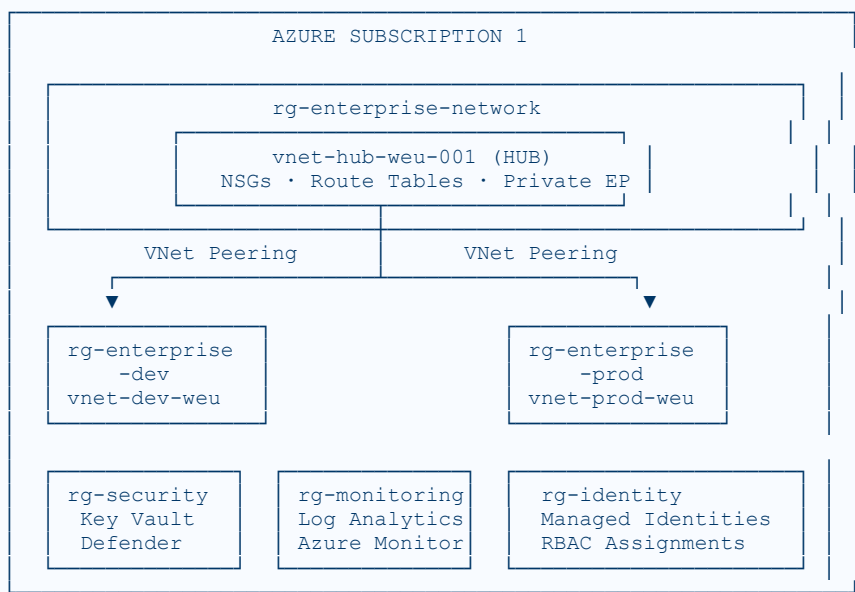
6. Architecture

6.1 High-Level Architecture

The high-level architecture diagram below illustrates the overall topology of the environment. The design places the hub Virtual Network at the centre of all network communications, with spoke VNets for development and production environments connected via VNet Peering.

Architecture Pattern: Hub & Spoke Landing Zone

The Hub & Spoke model is the recommended enterprise network topology for Azure. The hub acts as a central transit point for all traffic, enabling centralised security inspection, logging, and policy enforcement. Spoke VNets are isolated by default and only communicate through the hub.



6.2 Security Architecture

Security is implemented as a layered model, often referred to as Defence in Depth. Each layer provides independent controls, meaning that a failure in one layer does not compromise the entire environment. The layers from outermost to innermost are: Subscription Policy, Network Perimeter (NSGs/ASGs), Identity (RBAC/Managed Identity), Application (Key Vault secrets), and Data.

7. Resource Groups

7.1 Introduction

Resource Groups are the fundamental organisational unit in Azure. They serve as logical containers that hold related resources for an Azure solution, and they define the management boundary for access control, billing, and lifecycle management. In an enterprise environment, Resource Groups are designed to group resources that share the same lifecycle and that require the same governance policies.

7.2 Objective

The Resource Group strategy adopted in this project follows a functional segmentation model, where each group is aligned to a specific domain of responsibility. This design enables granular Role-Based Access Control (RBAC) assignments, per-group budget tracking, and independent lifecycle management — resources in one group can be deleted or redeployed without affecting other domains.

7.3 Implementation

Seven Resource Groups were created across the subscription, all deployed to the West Europe region. The naming convention follows the pattern `rg-<domain>-<environment>`, ensuring immediate identification of each group's purpose:

- ▶ NetworkWatcherRG — Automatically created by Azure Network Watcher; used for network diagnostics
- ▶ rg-enterprise-dev — Development environment workloads and virtual machines
- ▶ rg-enterprise-identity — Managed Identities and identity-related resources
- ▶ rg-enterprise-monitoring — Log Analytics Workspace and monitoring infrastructure
- ▶ rg-enterprise-network — All networking resources including VNets, NSGs, Route Tables
- ▶ rg-enterprise-prod — Production environment workloads
- ▶ rg-enterprise-security — Key Vault, Microsoft Defender for Cloud, security policies
- ▶ rg-secure-landing-zone — Secure Landing Zone baseline components

7.4 Benefits

- ▶ Granular Access Control — RBAC can be assigned at Resource Group level, limiting blast radius
- ▶ Simplified Cost Attribution — each group's costs appear independently in Cost Management
- ▶ Lifecycle Isolation — deleting a dev environment does not affect production resources
- ▶ Policy Scope — Azure Policies can be applied at subscription or Resource Group level
- ▶ Tagging Compliance — tags applied at group level propagate to child resources

7.5 Screenshot — Resource Groups Overview

The following screenshot confirms the successful deployment of all Resource Groups within the Azure subscription, each correctly assigned to the West Europe region.

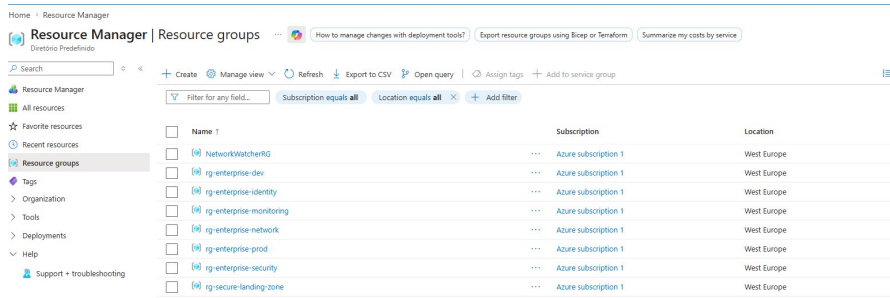


Figure 7.1 — Azure Portal: Resource Manager showing all 8 Resource Groups deployed in West Europe

7.6 Conclusion

The Resource Group structure forms the organisational backbone of the environment. Every subsequent resource deployed in this project is placed in the appropriate Resource Group according to its functional domain. This structure directly supports the principles of least privilege and separation of concerns that underpin the security model of the Landing Zone.

8. Networking

8.1 Introduction

The network layer is the primary security boundary in any cloud environment. Azure Virtual Networking enables the creation of isolated, private network environments that replicate on-premises networking concepts including subnets, routing, and security controls, while adding cloud-native capabilities such as VNet Peering, Private Endpoints, and Service Endpoints.

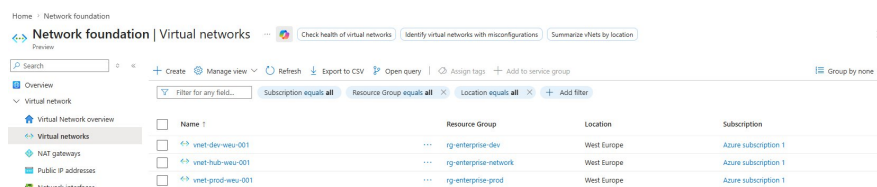
8.2 Objective

The networking objective was to implement a Hub & Spoke topology that centralises all network inspection and policy enforcement in the hub VNet, while providing isolated connectivity for development and production workloads in their respective spoke VNets. Traffic between spokes must flow through the hub, and internet-bound traffic is controlled and audited.

8.3 Virtual Networks

Three Virtual Networks were deployed across the three workload Resource Groups:

- ▶ vnet-hub-weu-001 (rg-enterprise-network) — Hub VNet; hosts shared services, Private Endpoints, and security appliances
- ▶ vnet-dev-weu-001 (rg-enterprise-dev) — Development spoke; isolated development workloads
- ▶ vnet-prod-weu-001 (rg-enterprise-prod) — Production spoke; production workloads with stricter controls



Name	Resource Group	Location	Subscription
vnet-dev-weu-001	rg-enterprise-dev	West Europe	Azure subscription 1
vnet-hub-weu-001	rg-enterprise-network	West Europe	Azure subscription 1
vnet-prod-weu-001	rg-enterprise-prod	West Europe	Azure subscription 1

Figure 8.1 — Azure Portal: Virtual Networks overview showing the Hub, Dev, and Prod VNets in West Europe

8.4 Network Security Groups

Network Security Groups (NSGs) function as stateful packet filters, allowing or denying inbound and outbound traffic to Azure resources based on source, destination, port, and protocol. Six NSGs were deployed following a tiered naming convention that reflects their scope and purpose.

The naming convention adopted was `nsg-<environment>-<tier>`, providing immediate clarity about each NSG's location in the network hierarchy. NSGs were associated with both subnets and network interfaces where applicable, providing layered filtering.

Name	Resource Group	Location	Subscription	Flow log
ns-g-dev-app	rg-enterprise-security	West Europe	Azure subscription 1	
ns-g-dev-data	rg-enterprise-security	West Europe	Azure subscription 1	
ns-g-management	rg-enterprise-security	West Europe	Azure subscription 1	
ns-g-private-subnet	rg-secure-landing-zone	West Europe	Azure subscription 1	
ns-g-prod-app	rg-enterprise-security	West Europe	Azure subscription 1	
ns-g-prod-data	rg-enterprise-security	West Europe	Azure subscription 1	

Figure 8.2 — Azure Portal: Network Security Groups showing *ns-g-dev-app*, *ns-g-dev-data*, *ns-g-management*, *ns-g-private-subnet*, *ns-g-prod-app*, and *ns-g-prod-data*

8.5 Application Security Groups

Application Security Groups (ASGs) are a higher-level abstraction over NSGs that allow administrators to define and reference groups of virtual machines by their application role rather than by individual IP addresses. This significantly simplifies rule management in dynamic environments where VM IP addresses may change.

Four ASGs were created: *asg-app* (application tier), *asg-db* (database tier), *asg-security* (security appliances), and *asg-web* (web tier). These groups can be referenced directly in NSG rules, making security policy language closer to application logic.

Name	Network interfaces count	Virtual network	Type	Resource Group	Location	Subscription
asg-app	0	-	Application security group	rg-enterprise-network	West Europe	Azure subscription 1
asg-db	0	-	Application security group	rg-enterprise-network	West Europe	Azure subscription 1
asg-security	1	vnet-hub-weu-001	Application security group	rg-enterprise-security	West Europe	Azure subscription 1
asg-web	0	-	Application security group	rg-enterprise-network	West Europe	Azure subscription 1

Figure 8.3 — Azure Portal: Application Security Groups — *asg-app*, *asg-db*, *asg-security*, and *asg-web*

8.6 Route Tables

Route Tables (User Defined Routes — UDRs) override Azure's default system routing to enforce that traffic takes specific paths through the network. In a Hub & Spoke architecture, UDRs are essential to ensure that spoke-to-spoke traffic transits through the hub, enabling centralised inspection.

Three Route Tables were deployed: *rt-hub-weu-001* (hub routing), *rt-dev-weu-001* (dev spoke), and *rt-prod-weu-001* (prod spoke). Each route table contains routes that direct inter-spoke traffic through the hub VNet's Network Virtual Appliance or Azure Firewall subnet.

Name	Resource Group	Location	Subscription
rt-dev-weu-001	rg-enterprise-network	West Europe	Azure subscription 1
rt-hub-weu-001	rg-enterprise-network	West Europe	Azure subscription 1
rt-prod-weu-001	rg-enterprise-network	West Europe	Azure subscription 1

Figure 8.4 — Azure Portal: Route Tables — *rt-dev-weu-001*, *rt-hub-weu-001*, and *rt-prod-weu-001* in *rg-enterprise-network*

8.7 Benefits

- ▶ Traffic Segmentation — NSGs enforce micro-segmentation at subnet and NIC level
- ▶ Application-Aware Security — ASGs allow policy rules based on workload role
- ▶ Centralised Inspection — UDRs enforce hub transit, enabling unified security monitoring
- ▶ East-West Traffic Control — lateral movement between environments is restricted by design
- ▶ Private Connectivity — Private Endpoints bring PaaS services into the private network

8.8 Conclusion

The networking layer provides a robust, multi-layered security boundary that enforces traffic segmentation, controls routing, and enables centralised inspection. The Hub & Spoke topology is aligned with Microsoft's Cloud Adoption Framework and provides a scalable foundation for adding additional spoke environments as the organisation grows.

9. Identity & Access Management

9.1 Introduction

Identity is the new perimeter in cloud security. Microsoft's Zero Trust model places identity verification at the core of every access decision, eliminating the assumption that anything inside the network perimeter is trusted. Azure's identity platform — Microsoft Entra ID (formerly Azure Active Directory) — provides the authentication and authorisation framework for all Azure resources.

9.2 Objective

The IAM objective was to implement the principle of least privilege across all resources, ensuring that every principal — whether a human user or a workload identity — has only the permissions required to perform its specific function. This was achieved through a combination of Azure RBAC role assignments and Managed Identities.

9.3 Implementation

RBAC role assignments were scoped at the most restrictive level possible. Rather than assigning broad Owner or Contributor roles at subscription level, roles were assigned at Resource Group or individual resource level. The Key Vault Access Control configuration below illustrates this principle in practice.

A Managed Identity (mi-enterprise-app) was created and assigned the Key Vault Secrets User role, scoped exclusively to the kv-enterprise-weu-001 Key Vault resource. This means the managed identity can read secrets from this specific vault only — it cannot read keys, certificates, or secrets from any other vault.

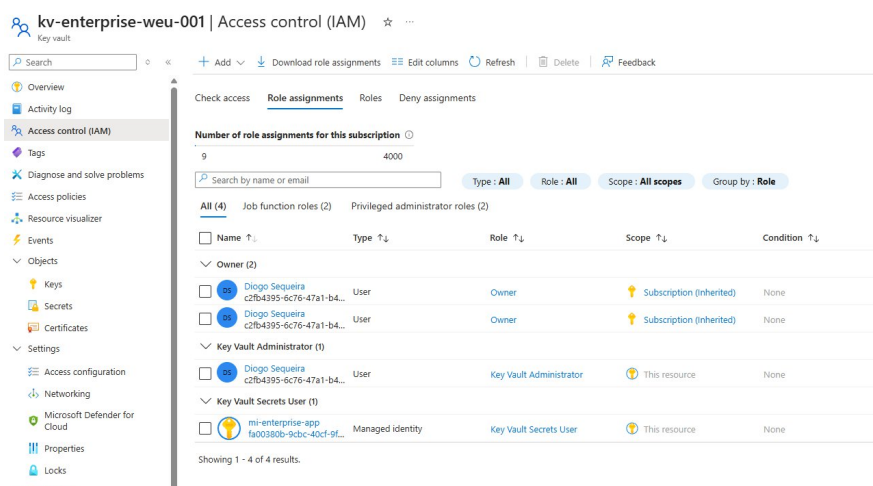


Figure 9.1 — Key Vault Access Control (IAM): Role assignments showing Owner (Diogo Sequeira), Key Vault Administrator (Diogo Sequeira), and Key Vault Secrets User (mi-enterprise-app Managed Identity)

9.4 Managed Identities

Managed Identities eliminate the need to store credentials in application code or configuration files. Instead, Azure manages the identity lifecycle automatically, rotating credentials transparently. The mi-enterprise-app managed identity was used to demonstrate secure, credential-free access from an application workload to the Key Vault — a pattern that forms the basis of the Key Vault access flow described in Chapter 11.

9.5 Benefits

- ▶ Least Privilege — roles scoped at resource level minimise blast radius of credential compromise
- ▶ No Credential Management — Managed Identities eliminate hardcoded secrets in code
- ▶ Auditability — every role assignment is recorded in the Azure Activity Log
- ▶ Separation of Duties — Administrator and Secrets User roles are distinct and non-overlapping
- ▶ Zero Trust Alignment — identity verification is required for every access, from every location

9.6 Conclusion

The IAM configuration demonstrates enterprise-grade access control practices. The combination of fine-grained RBAC, managed identities, and Key Vault RBAC creates a complete, auditable access control chain from workload identity to secret retrieval, without requiring any stored credentials.

10. Governance

10.1 Introduction

Cloud governance refers to the set of rules, processes, and controls that define how cloud resources are provisioned, configured, and managed. Without governance, cloud environments become inconsistent, difficult to audit, and prone to security and compliance failures. Azure Policy is the primary governance mechanism in Azure, enabling automated enforcement of organisational standards.

10.2 Objective

The governance objective was to establish a baseline of controls that enforce consistent resource configuration across the entire subscription. This included tagging requirements, allowed regions, required security configurations, and diagnostic settings. Governance ensures that new resources deployed into the environment automatically conform to the established security baseline.

10.3 Implementation

Azure Policy definitions were assigned at the subscription level, ensuring they apply to all Resource Groups and all resources within the subscription. Key policies implemented include:

- ▶ Require a tag on resources — all resources must have an Environment tag (dev, test, or prod) and an Owner tag
- ▶ Allowed locations — resources may only be deployed in West Europe to ensure data residency
- ▶ Require diagnostic settings — all applicable resources must have diagnostic settings configured to send logs to the Log Analytics workspace
- ▶ Key Vault should have soft delete enabled — enforces recovery protection for Key Vault instances
- ▶ Audit VMs without managed disks — identifies non-compliant VM configurations

Policy Effect: Audit vs. Deny

Azure Policy supports multiple effects. The Audit effect generates a compliance report but does not block resource creation. The Deny effect prevents non-compliant resources from being deployed. In this environment, tagging and location policies use Deny, while diagnostic settings policies use AuditIfNotExists to detect gaps in monitoring coverage.

10.4 Benefits

- ▶ Automated Compliance — policies enforce standards without manual intervention
- ▶ Consistent Tagging — tags enable cost attribution and resource inventory queries
- ▶ Regional Compliance — location policies enforce data residency requirements
- ▶ Security Baseline — policies ensure diagnostic settings are configured on all services
- ▶ Audit Trail — policy compliance state is visible in the Azure Portal and exportable

10.5 Conclusion

Governance via Azure Policy transforms compliance from a manual audit process into an automated, continuous enforcement mechanism. The policies implemented in this

environment form a security baseline that future resource deployments must satisfy, ensuring the environment remains compliant as it evolves.

11. Security

11.1 Introduction

The security chapter covers the implementation of Azure Key Vault as the central secret management service for the environment. Key Vault is a Hardware Security Module-backed service that enables secure storage and access to secrets, encryption keys, and certificates. When combined with Private Endpoints, Key Vault becomes accessible only from within the private network, eliminating exposure to the public internet.

11.2 Objective

The security objective was to implement a complete, production-grade secret management pipeline: applications request secrets via their Managed Identity, the identity is authenticated by Microsoft Entra ID, access is authorised by Key Vault RBAC, the request transits the private network via a Private Endpoint, and the secret is returned over an encrypted TLS channel — without any secret ever appearing in application code or environment variables.

11.3 Key Vault Implementation

Azure Key Vault kv-enterprise-weu-001 was deployed in the rg-enterprise-security Resource Group. The vault was configured with the following security settings: RBAC authorisation model (not legacy Access Policies), soft delete enabled for 90-day recovery window, purge protection enabled, and public network access disabled (private endpoint only).

11.4 Private Endpoint

A Private Endpoint (pe-kv-weu-001) was created to bring the Key Vault into the vnet-hub-weu-001 Virtual Network. The private endpoint creates a private network interface within the hub subnet, assigned a private IP address from the VNet address space. A Private DNS Zone was created automatically to resolve kv-enterprise-weu-001.vault.azure.net to the private IP address for all resources within the VNets.

The connection state shown below confirms the private endpoint is Approved, meaning Key Vault has accepted and established the private connection. The sub-resource type vault confirms this is a Key Vault-specific private link connection.

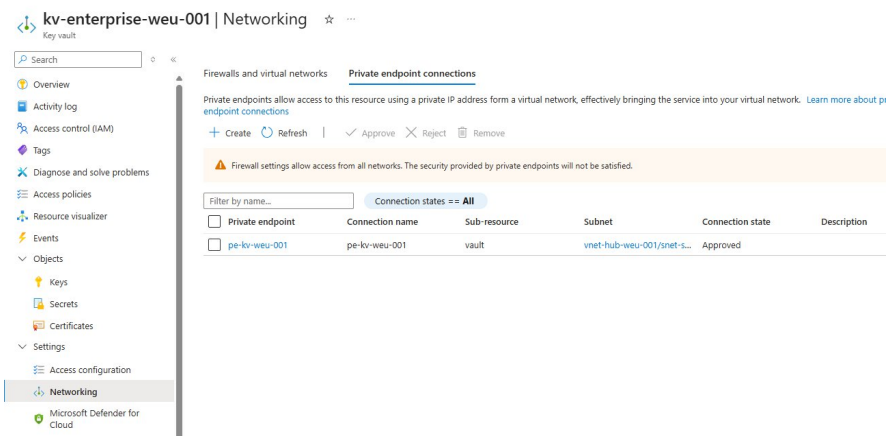
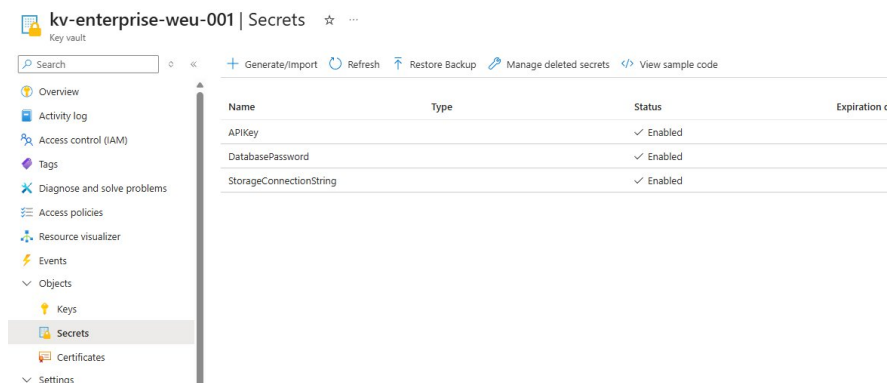


Figure 11.1 — Key Vault Networking: Private Endpoint Connections tab showing pe-kv-weu-001 connected to vnet-hub-weu-001 with Approved status

11.5 Secrets Management

Three secrets were provisioned in the Key Vault to simulate a realistic application secret management scenario: APIKey (a third-party API authentication key), DatabasePassword (a database connection credential), and StorageConnectionString (an Azure Storage Account connection string). All secrets are in Enabled state, meaning they are active and retrievable by authorised identities.

In a production scenario, these secrets would be referenced by application code via the Azure SDK or Azure App Configuration, using the Managed Identity assigned in Chapter 9, without any hardcoded values.

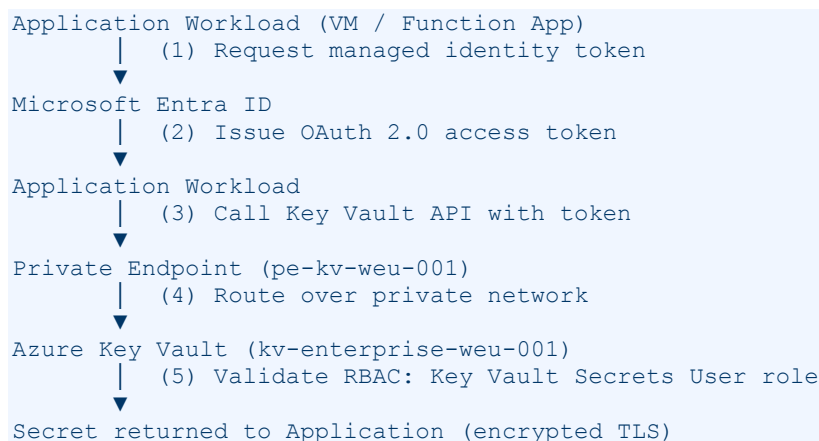


Name	Type	Status	Expiration d
APIKey		✓ Enabled	
DatabasePassword		✓ Enabled	
StorageConnectionString		✓ Enabled	

 Figure 11.2 — Key Vault Secrets: APIKey, DatabasePassword, and StorageConnectionString — all Enabled

11.6 Key Vault Access Flow

The following diagram illustrates the complete access flow for a workload retrieving a secret from Key Vault:



11.7 Benefits

- ▶ Zero Public Exposure — Private Endpoint ensures Key Vault is only accessible from within the private network
- ▶ No Hardcoded Secrets — Managed Identity eliminates credentials from application code
- ▶ Auditable Access — every secret retrieval is logged in Key Vault diagnostic logs
- ▶ Recovery Protection — soft delete and purge protection prevent accidental or malicious deletion
- ▶ RBAC Granularity — secrets, keys, and certificates have separate role definitions

11.8 Conclusion

The Key Vault implementation demonstrates a complete, production-grade secret management architecture. The combination of Private Endpoint, RBAC authorisation, and Managed Identity creates a zero-trust access model for sensitive credentials. No secret in this environment is stored in code, configuration files, or environment variables.

12. Monitoring

12.1 Introduction

Monitoring and observability are fundamental pillars of any secure, well-managed cloud environment. Without visibility into what is happening across the infrastructure, security incidents go undetected, performance degradation is missed, and compliance violations remain invisible. Azure provides a comprehensive observability stack centred on Azure Monitor and Log Analytics.

12.2 Objective

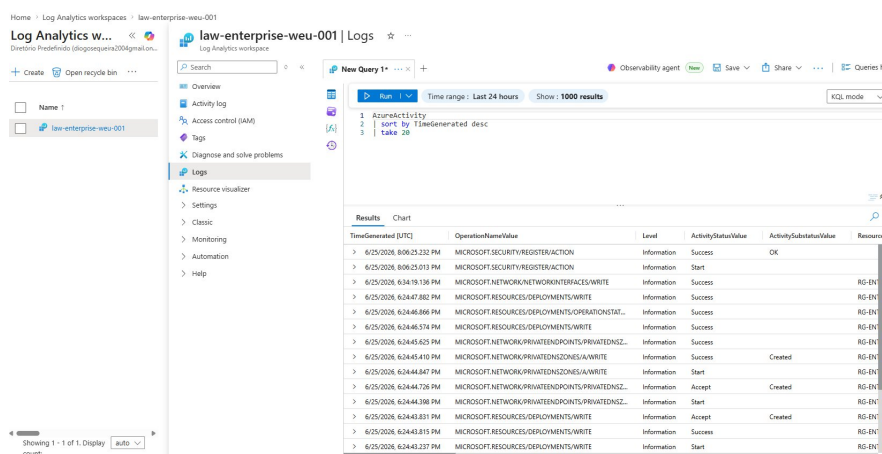
The monitoring objective was to establish a centralised logging and alerting platform that aggregates telemetry from all deployed services into a single queryable data plane. This enables security operations teams to perform incident investigation, compliance auditing, and operational troubleshooting from a single location using Kusto Query Language (KQL).

12.3 Log Analytics Workspace

A centralised Log Analytics Workspace (law-enterprise-weu-001) was deployed in the rg-enterprise-monitoring Resource Group. This workspace serves as the data sink for all diagnostic logs, metrics, and activity logs across the subscription. Diagnostic Settings were configured on all applicable resources to route their telemetry to this workspace.

12.4 Azure Activity Log via KQL

The screenshot below demonstrates an active KQL query against the Log Analytics workspace. The query retrieves the most recent 20 entries from the AzureActivity table, sorted by TimeGenerated in descending order. The results confirm that all recent Azure operations — including Private Endpoint creation, network interface writes, and deployment operations — are being captured and indexed.



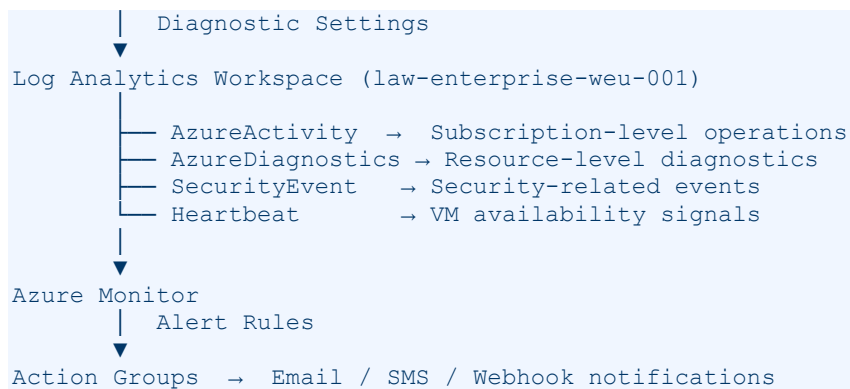
TimeGenerated [UTC]	OperationName/Value	Level	ActivityStatus/Value	ActivitySubStatus/Value	Resource
6/25/2026, 8:06:25.232 PM	MICROSOFT.SECURITY/REGISTRATION	Information	Success	OK	
6/25/2026, 8:06:25.013 PM	MICROSOFT.SECURITY/REGISTRATION	Information	Start		
6/25/2026, 6:34:19.136 PM	MICROSOFT.NETWORK/NETWORKINTERFACES/WRITE	Information	Success		RG-EN
6/25/2026, 6:24:47.862 PM	MICROSOFT.RESOURCES/DEPLOYMENTS/WRITE	Information	Success		RG-EN
6/25/2026, 6:24:46.866 PM	MICROSOFT.RESOURCES/DEPLOYMENTS/OPERATIONSTAT...	Information	Success		RG-EN
6/25/2026, 6:24:46.574 PM	MICROSOFT.RESOURCES/DEPLOYMENTS/WRITE	Information	Success		RG-EN
6/25/2026, 6:24:45.625 PM	MICROSOFT.NETWORK/PRIVATEENDPOINTS/PRIVATEENSG...	Information	Success		RG-EN
6/25/2026, 6:24:45.410 PM	MICROSOFT.NETWORK/PRIVATEENDPOINTS/PRIVATEENSG...	Information	Success	Created	RG-EN
6/25/2026, 6:24:44.847 PM	MICROSOFT.NETWORK/PRIVATEENDPOINTS/PRIVATEENSG...	Information	Start		RG-EN
6/25/2026, 6:24:44.726 PM	MICROSOFT.NETWORK/PRIVATEENDPOINTS/PRIVATEENSG...	Information	Accept	Created	RG-EN
6/25/2026, 6:24:44.388 PM	MICROSOFT.NETWORK/PRIVATEENDPOINTS/PRIVATEENSG...	Information	Start		RG-EN
6/25/2026, 6:24:43.831 PM	MICROSOFT.RESOURCES/DEPLOYMENTS/WRITE	Information	Accept	Created	RG-EN
6/25/2026, 6:24:43.815 PM	MICROSOFT.RESOURCES/DEPLOYMENTS/WRITE	Information	Success		RG-EN
6/25/2026, 6:24:43.237 PM	MICROSOFT.RESOURCES/DEPLOYMENTS/WRITE	Information	Start		RG-EN

 Figure 12.1 — Log Analytics Workspace law-enterprise-weu-001: KQL query on AzureActivity table showing recent MICROSOFT.NETWORK and MICROSOFT.RESOURCES operations

12.5 Monitoring Flow

The monitoring pipeline follows this data flow:

Azure Resources (VMs, Key Vault, NSGs, etc.)



12.6 KQL Sample Queries

The following KQL queries were used during the implementation to validate the monitoring pipeline and investigate activity:

```
// Recent activity log entries
AzureActivity
| sort by TimeGenerated desc
| take 20
```

12.7 Benefits

- ▶ Centralised Visibility — all telemetry flows to a single workspace, enabling cross-service correlation
- ▶ Security Incident Detection — activity logs enable detection of unauthorised or anomalous operations
- ▶ Compliance Auditing — complete audit trail of all resource changes exportable to SIEM systems
- ▶ Performance Monitoring — metrics from all services provide operational health dashboards
- ▶ Alert-Driven Operations — Azure Monitor alerts notify operations teams of critical conditions

12.8 Conclusion

The monitoring infrastructure provides complete observability across the environment. The combination of Log Analytics, Diagnostic Settings, and Azure Monitor transforms raw telemetry into actionable intelligence. The KQL query results confirm that the pipeline is fully operational, with all recent operations correctly captured and indexed.

13. Cost Management

13.1 Introduction

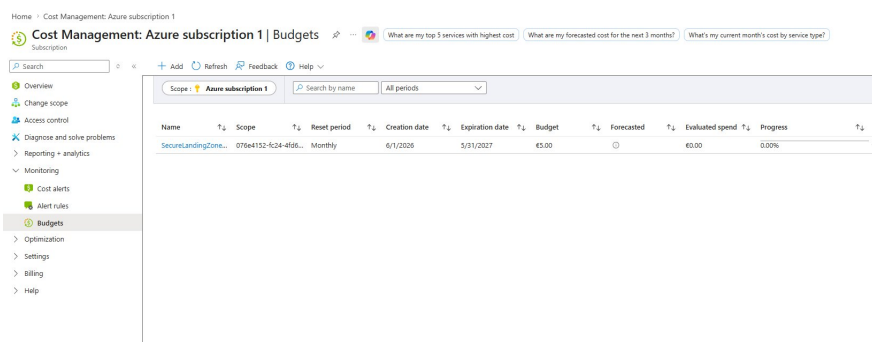
Cloud cost management is often overlooked during security-focused projects, but it is a critical operational discipline. Uncontrolled cloud spending can lead to project cancellation, and unexpected cost spikes may indicate security incidents such as cryptomining attacks or resource misuse. Azure Cost Management + Billing provides native tools to monitor, analyse, and control cloud expenditure.

13.2 Objective

The cost management objective was to implement proactive spending controls that alert responsible parties before budgets are exceeded, while also providing the visibility needed to attribute costs to specific workloads and environments. This is particularly important in a landing zone architecture, where multiple teams may deploy resources into the same subscription.

13.3 Implementation

An Azure Budget named SecureLandingZone... was created at the subscription scope (Azure subscription 1). The budget was configured with a monthly reset period, a €5.00 limit, a creation date of 01/06/2026, and an expiration date of 31/05/2027. Alert thresholds were configured at 80% (actual) and 100% (forecasted) of the budget limit, triggering email notifications to the subscription owner.



The screenshot shows the Azure Cost Management interface for 'Azure subscription 1'. The 'Budgets' section is active, displaying a table with one budget entry:

Name	Scope	Reset period	Creation date	Expiration date	Budget	Forecasted	Evaluated spend	Progress
SecureLandingZone...	076e4152-62d4-4b66...	Monthly	6/1/2026	5/31/2027	€5.00	0.00%	€0.00	0.00%

 Figure 13.1 — Azure Cost Management: Budget SecureLandingZone showing €5.00 monthly limit, 0.00% current spend, and monthly reset configuration

13.4 Benefits

- ▶ Proactive Alerting — email alerts prevent surprise bills at month end
- ▶ Security Signal — unexpected cost spikes can indicate unauthorised resource deployment
- ▶ Accountability — budgets create clear financial ownership at project level
- ▶ Forecasting — Azure provides spend forecasting to anticipate month-end totals
- ▶ Multi-scope Support — budgets can be applied at subscription, Resource Group, or resource level

13.5 Conclusion

The cost management controls ensure that this learning environment operates within defined financial boundaries. The €5.00 monthly budget reflects the scale of the

project while demonstrating the implementation pattern applicable to production environments with significantly larger budget thresholds.

14. Infrastructure as Code

14.1 Introduction

Infrastructure as Code (IaC) is the practice of managing and provisioning cloud infrastructure through machine-readable definition files, rather than through manual portal interactions. IaC transforms infrastructure from a manual, error-prone process into a version-controlled, repeatable, and auditable engineering discipline.

14.2 Objective

The IaC objective for this project is to convert all resources deployed manually through the Azure Portal into Terraform configuration files, organised into reusable modules. This enables the entire environment to be deployed, torn down, and redeployed consistently across different subscriptions and environments.

14.3 Planned Terraform Structure

The planned Terraform module structure follows a modular design, where each module encapsulates a specific infrastructure component:

```
terraform/
├── main.tf           # Root module – calls all child modules
├── variables.tf      # Input variables
├── outputs.tf        # Output values
├── providers.tf      # Azure provider configuration
├── terraform.tfvars  # Environment-specific values
├── modules/
│   ├── resource_groups/ # All Resource Group definitions
│   ├── networking/     # VNets, Subnets, NSGs, ASGs, Route Tables
│   ├── security/       # Key Vault, Private Endpoints, DNS Zones
│   ├── identity/       # Managed Identities, RBAC assignments
│   ├── monitoring/     # Log Analytics, Diagnostic Settings, Alerts
│   ├── governance/     # Azure Policy assignments
│   └── cost_management/ # Budgets and alert rules
```

14.4 GitHub Actions CI/CD Pipeline

The Terraform deployment will be automated using a GitHub Actions pipeline with the following stages: terraform fmt (code formatting validation), terraform validate (configuration validation), terraform plan (execution plan generation and review), and terraform apply (deployment to Azure, triggered on merge to main).

14.5 Benefits

- ▶ Repeatability — identical environments can be deployed from the same codebase
- ▶ Version Control — all infrastructure changes are tracked in Git with full history
- ▶ Peer Review — infrastructure changes undergo code review before deployment
- ▶ Documentation — Terraform code serves as self-documenting architecture
- ▶ Disaster Recovery — environments can be recreated from code in minutes

15. Challenges

15.1 Private Endpoint DNS Resolution

The most technically complex challenge encountered during implementation was configuring correct DNS resolution for the Key Vault Private Endpoint. When a Private Endpoint is created, Azure automatically creates a Private DNS Zone (privatelink.vaultcore.azure.net) and links it to the hub VNet. However, the DNS resolution only works correctly for resources within VNets that have the DNS zone linked.

For spoke VNets (dev and prod) to resolve the private DNS name, the Private DNS Zone link must be explicitly added for each spoke. This is a common misconfiguration in Hub & Spoke deployments and is easily overlooked during initial setup.

15.2 NSG Rule Ordering

Azure NSG rules are processed in priority order (lower number = higher priority). During initial configuration, a deny-all rule was inadvertently placed at a lower priority number than a required allow rule, blocking legitimate traffic. This was resolved by carefully reviewing rule priority assignments and testing connectivity after each change.

15.3 RBAC Propagation Delay

Azure RBAC role assignments have a propagation delay of up to 5-10 minutes after creation before they become effective. During testing, immediately attempting to access a resource after assigning a role resulted in access denied errors. Understanding this delay is important for operational runbooks and CI/CD pipelines that depend on RBAC assignments.

15.4 Key Vault Access Policy vs. RBAC

Azure Key Vault supports two authorisation models: legacy Access Policies and the newer Azure RBAC model. The RBAC model is recommended by Microsoft but requires enabling the Key Vault RBAC authorisation setting during vault creation. Switching between models after deployment requires careful planning to avoid access interruptions.

16. Lessons Learned

16.1 Architecture First, Implementation Second

The most valuable lesson from this project was the importance of designing the full architecture before deploying any resource. Resources deployed without a clear naming convention, tagging strategy, or network design required retrospective correction that was time-consuming and error-prone. In production environments, retrospective changes to foundational components like VNet address spaces or naming conventions can be prohibitively disruptive.

16.2 Zero Trust Is a Journey, Not a Destination

Implementing Zero Trust principles — never trust, always verify — revealed that security posture is continuously evolving. Each component added to the architecture introduces new attack surfaces that must be accounted for. The Private Endpoint for Key Vault, the RBAC least-privilege model, and the NSG micro-segmentation are each individually imperfect, but together they create a defence-in-depth posture that is significantly more resilient than any single control.

16.3 Monitoring Must Be Designed, Not Afterthought

Attempting to add monitoring retrospectively to a deployed environment is significantly harder than designing it in from the beginning. Diagnostic Settings must be configured on each resource individually if Azure Policy is not enforcing them automatically. This project demonstrated the value of implementing monitoring infrastructure (Log Analytics workspace, Diagnostic Settings policies) before deploying workload resources.

16.4 IaC Transforms Operations

Even though the initial deployment was performed manually through the Azure Portal, the process of documenting each step revealed the repetitive nature of manual infrastructure work. Terraform modules eliminate this repetition and reduce the risk of human error. The discipline of thinking in modules — reusable, parameterised components — directly improves the architectural quality of the deployed infrastructure.

16.5 Documentation Is Part of the Deliverable

A technical implementation without documentation is incomplete. This report, combined with the GitHub repository, transforms a collection of Azure resources into a demonstrable professional artefact. The act of writing documentation for each component also forces a deeper understanding of why each architectural decision was made — knowledge that is essential for technical interviews and client presentations.

17. Future Improvements

17.1 Azure Firewall

The current Hub & Spoke implementation uses NSGs and Route Tables for traffic control, but does not include an Azure Firewall. Adding an Azure Firewall to the hub VNet would provide stateful packet inspection, application-level filtering (FQDN-based rules), threat intelligence feed integration, and centralised logging of all east-west and north-south traffic. This is the natural next step for production hardening.

17.2 Microsoft Defender for Cloud

Microsoft Defender for Cloud (formerly Azure Security Center) provides continuous security posture assessment and advanced threat protection across Azure, on-premises, and multi-cloud environments. Enabling Defender for Cloud with enhanced security features would provide Secure Score tracking, regulatory compliance dashboards (ISO 27001, PCI DSS), and advanced threat detection for workloads.

17.3 Terraform Migration

The planned Terraform migration would convert all portal-deployed resources into version-controlled infrastructure code. This enables GitOps workflows where all infrastructure changes are proposed as pull requests, reviewed by peers, and applied automatically via CI/CD pipelines. The modular structure would also enable reuse across multiple environments.

17.4 Azure Sentinel (Microsoft Sentinel)

Microsoft Sentinel is a cloud-native SIEM (Security Information and Event Management) and SOAR (Security Orchestration, Automation and Response) platform built on top of Log Analytics. Connecting Sentinel to the existing Log Analytics workspace would enable advanced threat hunting, automated incident response playbooks, and built-in security analytics rules for common attack patterns.

17.5 Privileged Identity Management

Azure AD Privileged Identity Management (PIM) enables just-in-time privileged access, requiring administrators to request elevated permissions for a limited time window rather than holding permanent privileged roles. Implementing PIM would further reduce the attack surface of the IAM configuration by eliminating standing privileged access.

17.6 Multi-Region Redundancy

The current environment is deployed exclusively in West Europe. A production-grade architecture would include a secondary region (North Europe) for disaster recovery, with geo-redundant storage, Key Vault replication, and traffic manager failover policies.

18. Conclusion

This report has documented the complete design and implementation of an Enterprise Azure Security Environment, built according to Microsoft's Cloud Adoption Framework Landing Zone principles and Zero Trust security architecture. The environment demonstrates practical application of the core competencies required for Cloud Security Engineering roles, including network architecture, identity management, secret handling, governance, observability, and cost control.

The project produced three professional deliverables: a GitHub repository containing the architecture documentation, configuration files, and README; this technical report containing detailed implementation evidence and architectural reasoning; and the foundation for a LinkedIn publication documenting the project journey.

The technical choices made throughout the project — Hub & Spoke networking, NSG micro-segmentation, Private Endpoint for Key Vault, RBAC least privilege, centralised Log Analytics monitoring — are not academic exercises. They represent the same patterns applied by cloud engineering teams at organisations of all sizes, from scale-ups to global enterprises. The ability to design, implement, document, and explain these patterns is precisely what distinguishes a competent Cloud Security Engineer from a junior administrator.

The environment is designed as a living foundation: it can be extended with Azure Firewall, Microsoft Defender for Cloud, Microsoft Sentinel, and PIM as the next phases of implementation, directly mapped to the roadmap presented in Chapter 17.

Final Assessment

This project demonstrates production-grade Azure security architecture skills including: Hub & Spoke networking, Zero Trust identity, Key Vault + Private Endpoint integration, Azure RBAC least privilege, Log Analytics + KQL monitoring, Azure Policy governance, and cost management controls. All components are evidence-backed with Azure Portal screenshots and architectural documentation suitable for technical review by Cloud Security, DevSecOps, or Cloud Architecture hiring teams.

Appendix

A. Resource Naming Convention

All resources in this environment follow a consistent naming convention designed for readability, automation compatibility, and operational clarity:

Resource Type	Pattern	Example
Resource Group	rg-<domain>-<env>	rg-enterprise-security
Virtual Network	vnet-<loc>-<seq>	vnet-hub-weu-001
NSG	nsg-<env>-<tier>	nsg-prod-app
ASG	asg-<role>	asg-web
Route Table	rt-<env>-<loc>-<seq>	rt-hub-weu-001
Key Vault	kv-<domain>-<loc>-<seq>	kv-enterprise-weu-001
Private Endpoint	pe-<resource>-<loc>-<seq>	pe-kv-weu-001
Log Analytics	law-<domain>-<loc>-<seq>	law-enterprise-weu-001
Managed Identity	mi-<app>-<purpose>	mi-enterprise-app

B. Azure Services Reference

The following Azure services were used in this implementation:

- ▶ Azure Resource Groups — Logical containers for related Azure resources
- ▶ Azure Virtual Network (VNet) — Private network environment in Azure
- ▶ Network Security Group (NSG) — Stateful packet filter for network traffic
- ▶ Application Security Group (ASG) — Application-level network security grouping
- ▶ Route Table (UDR) — Custom routing to override Azure default routing
- ▶ Azure Key Vault — HSM-backed secret, key, and certificate management
- ▶ Private Endpoint — Private network access to Azure PaaS services
- ▶ Private DNS Zone — DNS resolution for Private Endpoint addresses
- ▶ Azure RBAC — Role-based access control for Azure resources
- ▶ Managed Identity — Azure-managed service identities without credentials
- ▶ Azure Policy — Governance and compliance automation
- ▶ Log Analytics Workspace — Centralised log data store and query engine
- ▶ Azure Monitor — Full-stack monitoring and alerting platform
- ▶ Kusto Query Language (KQL) — Query language for Log Analytics data
- ▶ Azure Cost Management — Cloud cost visibility and budget control

C. Useful KQL Queries

The following KQL queries are useful for operational and security tasks in this environment:

Failed RBAC operations

```
AzureActivity | where ActivityStatusValue == 'Failure' | where OperationNameValue contains 'ROLEASSIGNMENT' | sort by TimeGenerated desc
```

Key Vault secret access

```
AzureDiagnostics | where ResourceProvider == 'MICROSOFT.KEYVAULT' | where OperationName ==  
'SecretGet' | project TimeGenerated, CallerIPAddress, identity_claim appid s
```

Network traffic denied

```
AzureDiagnostics | where Category == 'NetworkSecurityGroupFlowEvent' | where flowStatus_s ==  
'D' | summarize count() by destinationAddress_s, destinationPort_d
```

Diogo Sequeira · Cloud Security Engineer · June 2026
Enterprise Azure Security Environment — Technical Implementation Report